

HTTP Analysis Using Wireshark - Text Traffic

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

8 September 2026

LAB OVERVIEW

This lab focused on generating, capturing, and forensically analyzing HTTP network traffic on a Kali Linux VM to understand how web communication works at the packet level, and how to preserve that evidence properly for investigative purposes.

I began by setting up a simple web page (**basic.html**) served locally through Apache2, then verified it was reachable and responding correctly using both a browser and `curl`. Once the web service was confirmed working, I used `tshark` to capture live traffic on the loopback interface while generating an HTTP request with `curl`, producing a `.pcapng` capture file containing the full client-server exchange. To maintain evidence integrity, I created a working copy of the capture and generated SHA-256 hashes for both the original and the copy, confirming no data had been altered during analysis, a core principle of digital forensic chain-of-custody.

From there, I analyzed the capture in detail: identifying the TCP three-way handshake (SYN, SYN-ACK, ACK), extracting the HTTP request and response headers (GET method, URI, Host, User-Agent, status code, server, content type), reconstructing the full request/response exchange, and tracing how sequence and acknowledgement numbers advanced in exact step with each packet's payload length. I also examined how data is encapsulated across the Ethernet, IP, TCP, and HTTP layers, and observed how the connection was gracefully closed through a FIN/ACK exchange rather than an abrupt reset. Finally, I noted that since the capture was taken over loopback, no genuine Ethernet MAC addresses were visible, since loopback traffic never actually touches a physical or virtual network interface.

OBJECTIVES

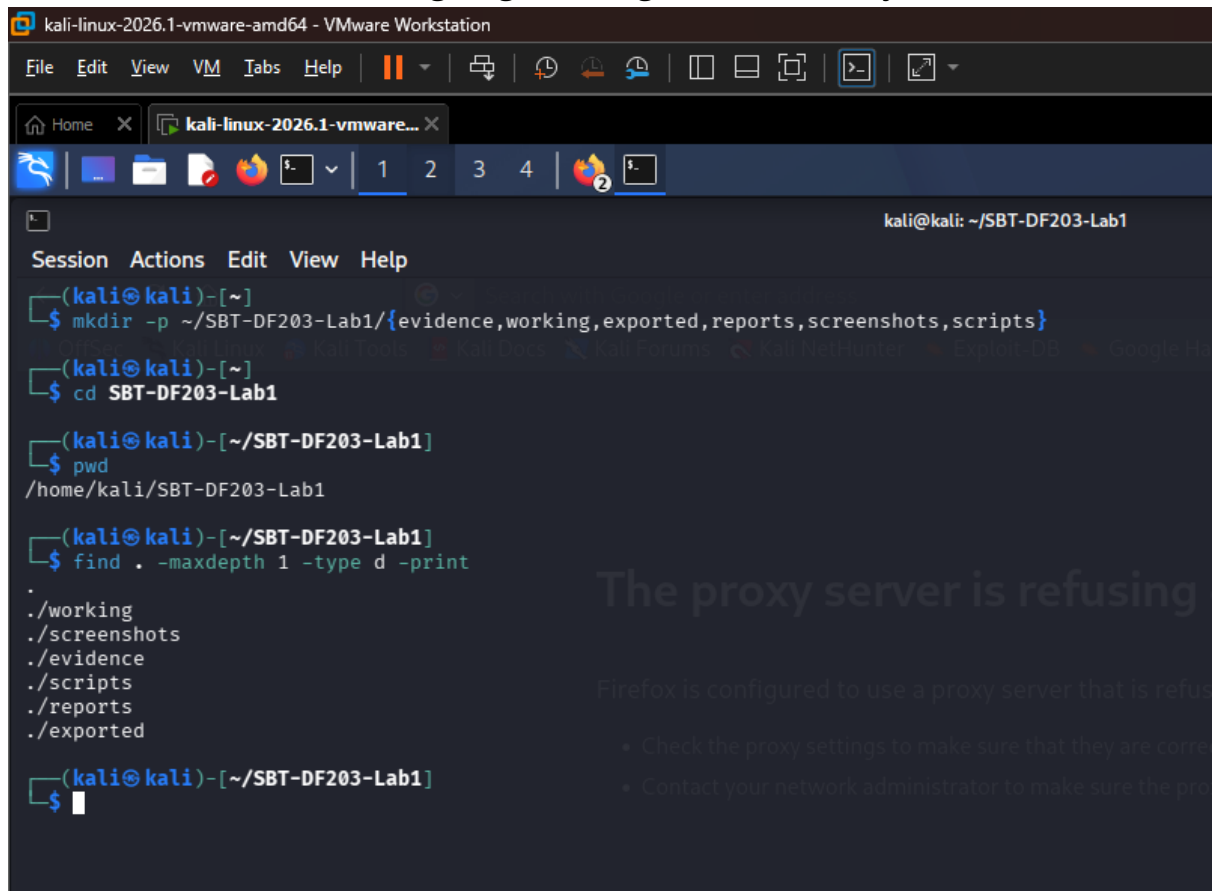
- Explain the forensic value of HTTP, TCP, IP and Ethernet metadata.
- Create and serve a simple HTML page using Apache.
- Capture HTTP traffic using Wireshark or `tshark` without altering the evidence.
- Identify the SYN, SYN-ACK and ACK packets in a TCP handshake.
- Extract source/destination ports, sequence numbers, acknowledgement numbers, timestamps and HTTP headers.
- Explain why loopback captures do not show normal Ethernet MAC addresses and repeat the capture on a two-VM network where required.
- Generate a concise network forensic timeline and evidence report

TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Github, Git
- `Md5sum` and `sha256sum`
- Apache2
- Wireshark, `tshark`, `Curl`, Web browser

METHODOLOGY

First thing is to create a working environment. Created the folder structure before downloading or generating evidence of any form.



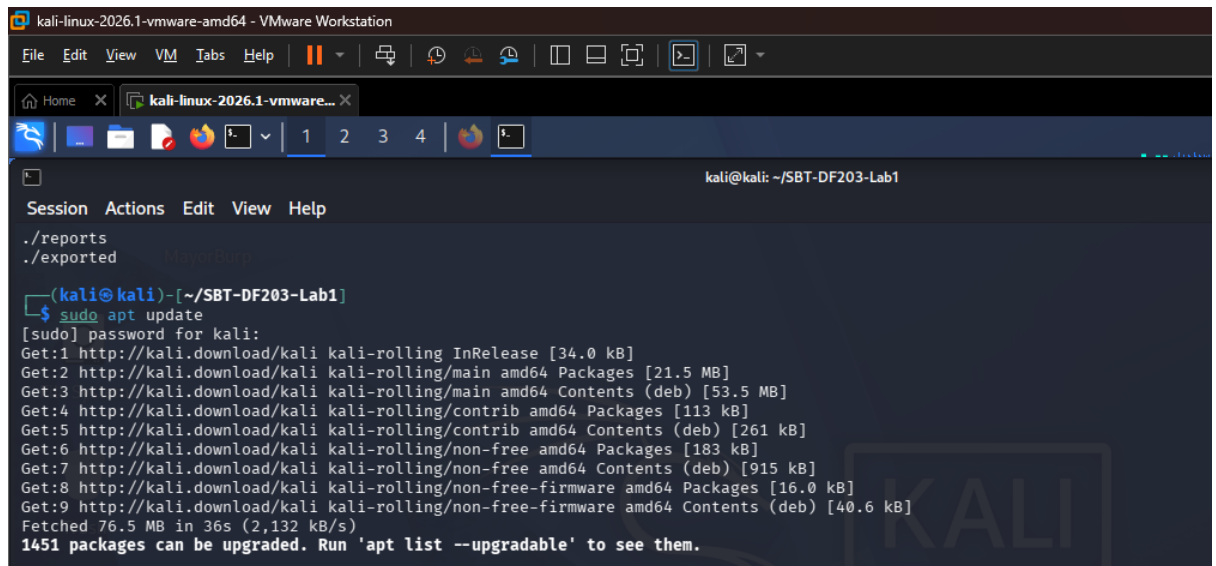
The screenshot shows a terminal window titled 'kali-linux-2026.1-vmware-amd64 - VMware Workstation'. The terminal output is as follows:

```
(kali@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
(kali@kali)-[~]
$ cd SBT-DF203-Lab1
(kali@kali)-[~/SBT-DF203-Lab1]
$ pwd
/home/kali/SBT-DF203-Lab1
(kali@kali)-[~/SBT-DF203-Lab1]
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported
(kali@kali)-[~/SBT-DF203-Lab1]
$
```

On the right side of the terminal window, there is a message: 'The proxy server is refusing' and 'Firefox is configured to use a proxy server that is refus'. Below this message, there are two bullet points: '• Check the proxy settings to make sure that they are corr' and '• Contact your network administrator to make sure the pro'.

Fig 1: Working Environment created

Then we update the sudo environment by using the `sudo apt update` command and then we install the necessary tool needed to run the lab like, apache (the server tool), wireshark, tshark which are used for capturing live network traffic (or reads a saved capture file) and allows to inspect every packet in detail.



```
kali@kali: ~/SBT-DF203-Lab1
Session Actions Edit View Help
./reports
./exported

(kali@kali)~[~/SBT-DF203-Lab1]
$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.5 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [261 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [40.6 kB]
Fetched 76.5 MB in 36s (2,132 kB/s)
1451 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Fig 2: Updating the Environment Package

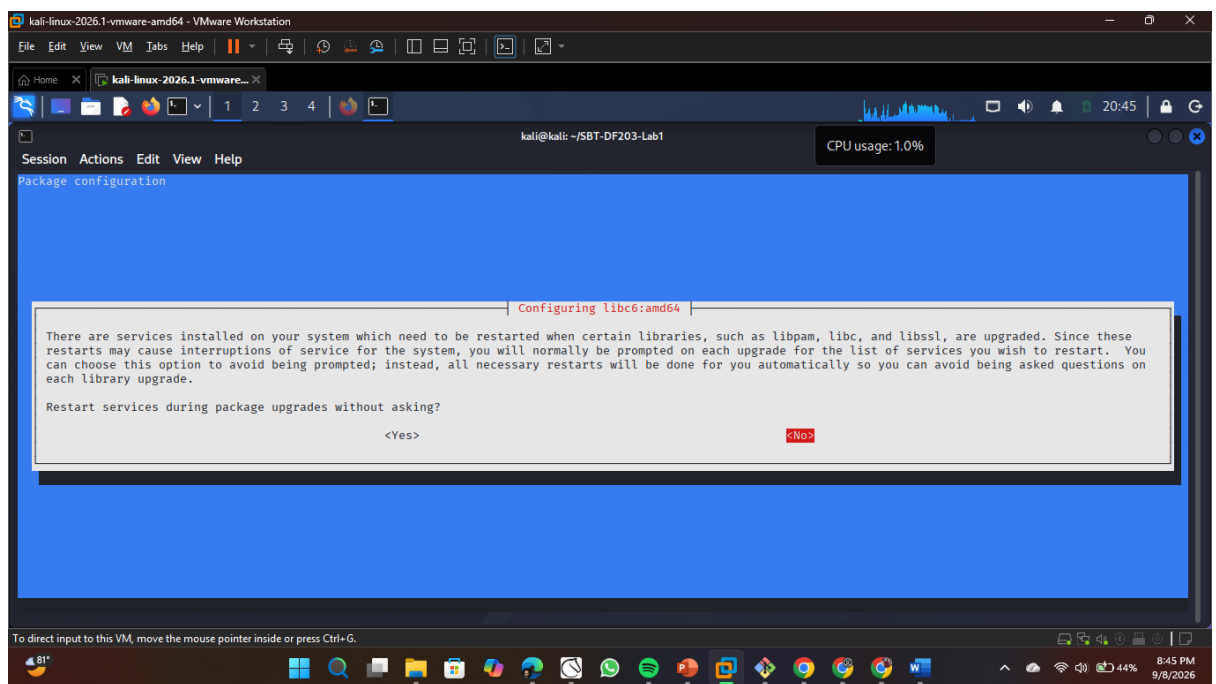


Fig 3: Picking yes for the configuration of Wireshark


```
(kali@kali)-[~/SBT-DF203-Lab1]
$ curl -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 48788
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 19:56:31 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 19:54:05 GMT
< ETag: "85-65afe18dfda2c"
< Accept-Ranges: bytes
< Content-Length: 133
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: OJE DOMINION MAYOWA</p><p>Reg No: FWSD2511507</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

(kali@kali)-[~/SBT-DF203-Lab1]
$ ss
```

Fig 7: Using Curl to Establish connection with the browser

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ ls
evidence  exported  reports  screenshots  scripts  working

(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -i lo -f "port 80" -w evidence/basic.pcapng
Capturing on 'Loopback: lo'
10 ^C
```

Fig 8: Opening a listener in the working environment using tshark

```
(kali@kali)-[~]
$ curl -v http://127.0.0.1/basic.html 20 OK
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 53492
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 20:11:39 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 19:54:05 GMT
< ETag: "85-65afe18dfda2c"
< Accept-Ranges: bytes
< Content-Length: 133
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: OJE DOMINION MAYOWA</p><p>Reg No: FWSD2511507</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

(kali@kali)-[~/SBT-DF203-Lab1]
$ ls
evidence/basic.pcapng  reports/basic_pcap_sha256.txt
```

Fig 9: sending file packets by curling the basic web site

```
(kali㉿kali)-[~/SBT-DF203-Lab1]
$ sha256sum evidence/basic.pcapng | tee reports/basic_pcap_sha256.txt
71341fe33f31270b6eefed1d4d681c3eda8fcc4b0073fe6306dfcb017434aafc evidence/basic.pcapng
```

Fig 7: Hashing the Original Capture

Then to keep the Integrity of the Original evidence we create a copy of the evidence by using the cp command

```
(kali㉿kali)-[~/SBT-DF203-Lab1]
$ cp evidence/basic.pcapng evidence/basic_workingcopy.pcapng

(kali㉿kali)-[~/SBT-DF203-Lab1]
$ ls -lh evidence/basic_workingcopy.pcapng
-rw-r--r-- 1 kali kali 1.8K Sep  8 21:19 evidence/basic_workingcopy.pcapng
```

Fig 8: Creating a working copy of the Original Evidence

Then we also check the Integrity of the copy by hashing it also

```
(kali㉿kali)-[~/SBT-DF203-Lab1]
$ sha256sum evidence/basic_workingcopy.pcapng | tee reports/basic_pcap_sha256_workingcopy.txt
71341fe33f31270b6eefed1d4d681c3eda8fcc4b0073fe6306dfcb017434aafc evidence/basic_workingcopy.pcapng
```

Fig 8: Creating a working copy of the Original Evidence

From the figure above we see that the Integrity of the evidence is intact as the Hashed values of both the Original and Copy are the same thing.

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab1-OJE-DOMINION
Trainee name	OJE DOMINION
Date and time started	08/09/2026, 5:00pm WAT

Field	Student Entry
Evidence file name(s)	
Source or generation method	tshark -i lo -f "port 80" -w evidence/basic.pcapng
Original SHA-256	71341fe33f31270b6eefed1d4d681c3eda8fcc4b0073fe6306dfcb017434aafc
Working-copy SHA-256	71341fe33f31270b6eefed1d4d681c3eda8fcc4b0073fe6306dfcb017434aafc
Analysis workstation/VM	
Notes on any changes	

Part A - Verify the Web Service and Generate Text HTTP Traffic

The name and placeholders in the basic.html file have been replaced with my details

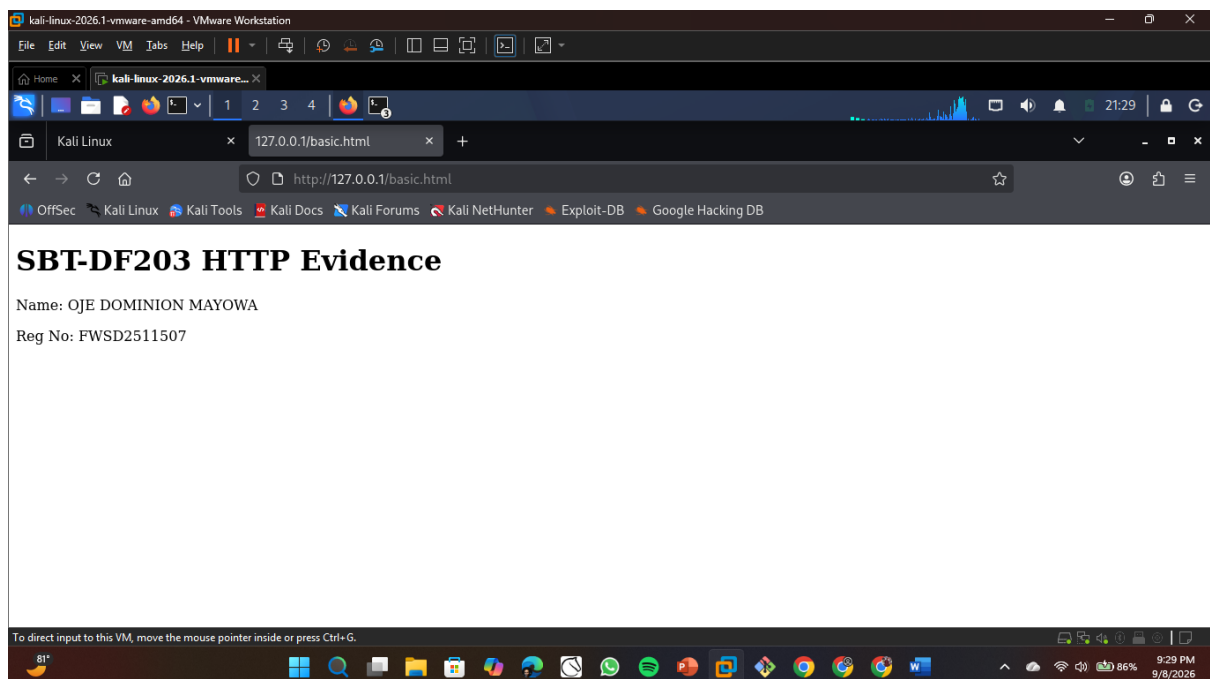


Fig 9: Basic.html file confirmed with necessary details


```
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo systemctl status apache2 --no-pager
[sudo] password for kali:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-09-08 20:47:27 WAT; 43min ago
     Invocation: 4d1b65e359b14281be2637d40ba9830d
   Main PID: 902037 (apache2)
  Reg Status: "Total requests: 4; Idle/Busy workers 100/0;Requests/sec: 0.00152; Bytes served/sec: 1 B/sec"
    Tasks: 7 (limit: 2069)
   Memory: 31.4M (peak: 32.1M)
      CPU: 908ms
   CGroup: /system.slice/apache2.service
           └─002037 /usr/sbin/apache2 -k start -DFOREGROUND
             └─002040 /usr/sbin/apache2 -k start -DFOREGROUND
               └─002041 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─002042 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─002044 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─002045 /usr/sbin/apache2 -k start -DFOREGROUND
                       └─923238 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 08 20:47:27 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 08 20:47:27 kali apachectl[902037]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the... this message
Sep 08 20:47:27 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

(kali@kali)-[~/SBT-DF203-Lab1]
```

Fig 10: Apache Server Confirmed Active and running

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ sudo ss -ltnp | grep ':80'
LISTEN 0      511          *:*          *:*    users:((("apache2",pid=923238,fd=4),("apache2",pid=902045,fd=4),("apache2",pid=902044,fd=4),("apache2",pid=902042,fd=4),("apache2",pid=902041,fd=4),("apache2",pid=902040,fd=4),("apache2",pid=902037,fd=4)))

(kali@kali)-[~/SBT-DF203-Lab1]
```

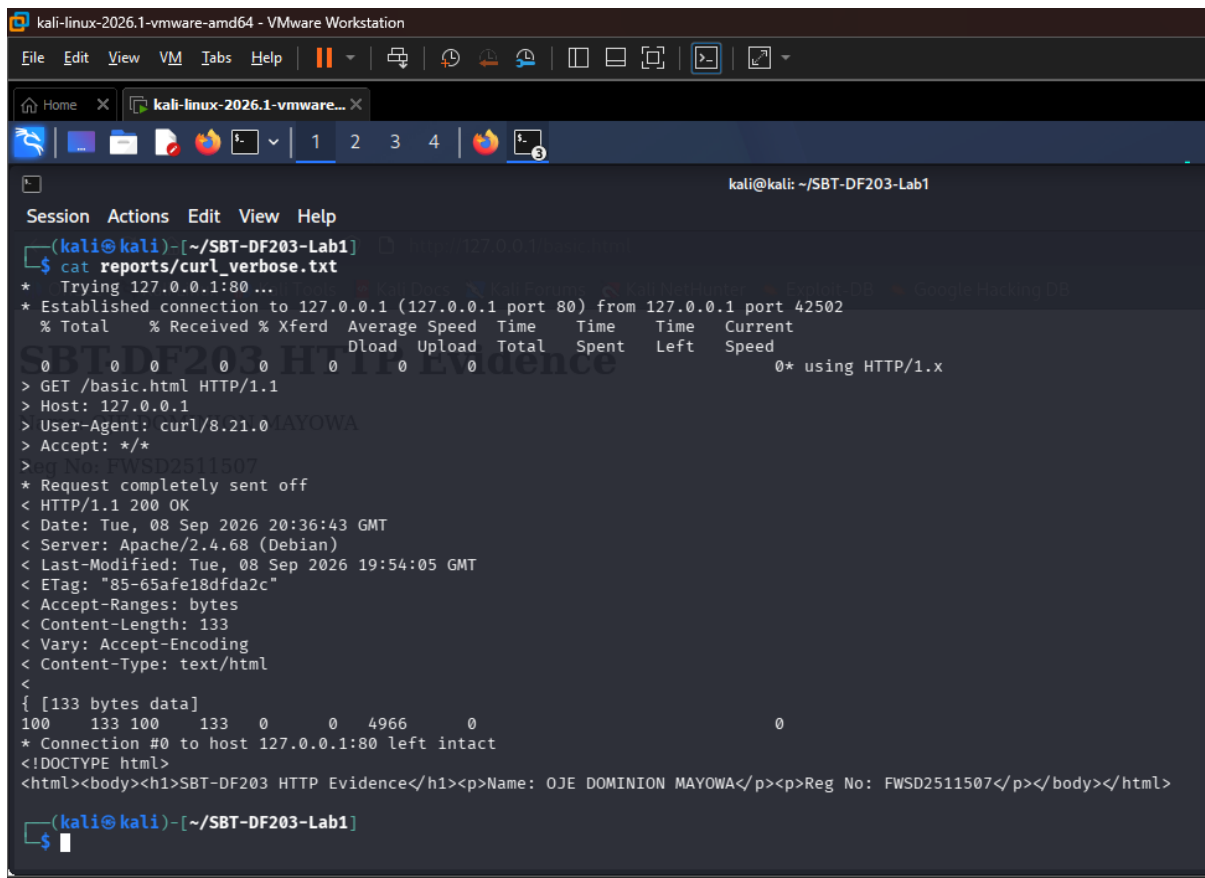
Fig 11: Confirms Apache is bound to Port 80 and is listening
The local service state is 511 and the listening port is port 80

```
kali@kali: ~/SBT-DF203-Lab1
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab1]
$ curl -v http://127.0.0.1/basic.html 2>&1 | tee reports/curl_verbose.txt
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 42502
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
                                 Dload  Upload  Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0      0     0  0     0      0
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0 MAYOWA
> Accept: */*
> Reg No: FWSD2511507
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 20:36:43 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 19:54:05 GMT
< ETag: "85-65afe18dfda2c"
< Accept-Ranges: bytes
< Content-Length: 133
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [133 bytes data]
100  133 100   133    0    0  4966    0      0
* Connection #0 to host 127.0.0.1:80 left intact
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: OJE DOMINION MAYOWA</p><p>Reg No: FWSD2511507</p></body></html>

(kali@kali)-[~/SBT-DF203-Lab1]
```

Fig 12 Using curl on the basic.html to get the response and request header

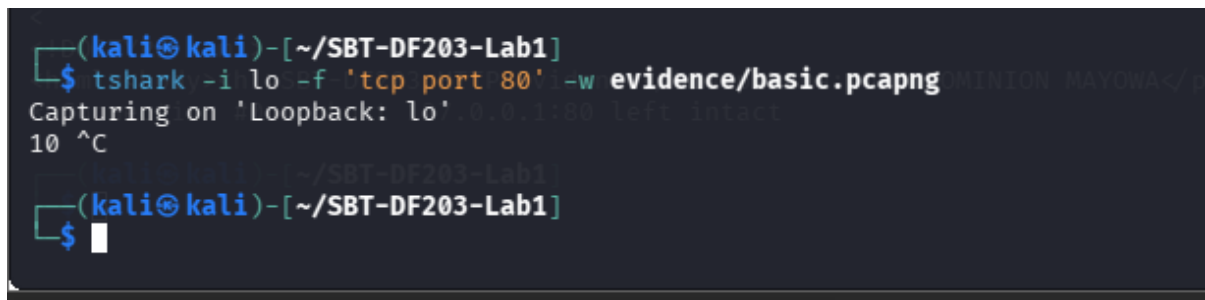
Now checking the saved response and request headers using the cat command on the curl_verbose.txt file where they were saved.



```
kali-linux-2026.1-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.1-vmware...
kali@kali: ~/SBT-DF203-Lab1
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab1]
$ cat reports/curl_verbose.txt
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 42502
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0    0     0    0    0    0     0      0      0  0* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0 MAYOWA
> Accept: */*
> Reg No: FWSD2511507
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 20:36:43 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 19:54:05 GMT
< ETag: "85-65afe18dfa2c"
< Accept-Ranges: bytes
< Content-Length: 133
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [133 bytes data]
100  133 100  133  0    0  4966    0
* Connection #0 to host 127.0.0.1:80 left intact
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: OJE DOMINION MAYOWA</p><p>Reg No: FWSD2511507</p></body></html>
(kali@kali)-[~/SBT-DF203-Lab1]
$
```

Fig 13: Confirms Apache is bound to Port 80 and is listening

Part B - Capture the Session



```
(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
Capturing on 'Loopback: lo'
10 ^C
(kali@kali)-[~/SBT-DF203-Lab1]
$
```

Fig 14: A Listening Port was opened using tshark

We then opened another terminal inside the same working directory to be able to request

```
kali@kali: ~/SBT-DF203-Lab1
Session Actions Edit View Help
(kali@kali)-[~]
$ cd SBT-DF203-Lab1
(kali@kali)-[~/SBT-DF203-Lab1]
$ curl --no-keepalive -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 49640
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 20:48:23 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 19:54:05 GMT
< ETag: "85-65afe18dfa2c"
< Accept-Ranges: bytes
< Content-Length: 133
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: OJE DOMINION MAYOWA</p><p>Reg No: FWS02511507</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact
(kali@kali)-[~/SBT-DF203-Lab1]
$
```

Fig 15: A Listening Port was opened using tshark

```
kali@kali: ~/SBT-DF203-Lab1
Session Actions Edit View Help
* Connection #0 to host 127.0.0.1:80 left intact
(kali@kali)-[~/SBT-DF203-Lab1]
$ ls -lh evidence/basic.pcapng
-rw-r--r-- 1 kali kali 1.0K Sep  8 21:48 evidence/basic.pcapng
(kali@kali)-[~/SBT-DF203-Lab1]
$ cat evidence/basic.pcapng

*****6Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz (with SSE4.2)Linux 6.19.14-kali-amd64Dumpcap (Wireshark) 4.6.6*Loopback

tcp port 80
Linux 6.19

.14+kali-amd64*JECi*PN#*****
ph*ll*s*JEC*P*QK*PN#*****
xE*ph*ld*s*BE4i*PN*QK*
ph*cx*E*d*s*BE4i*PN*QK*
ph*cx*E*GET /basic.html HTTP/1.1
Host: 127.0.0.1
User-Agent: curl/8.21.0
Accept: */*
* Request completely sent off
* Connection #0 to host 127.0.0.1:80 left intact
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 20:48:23 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 19:54:05 GMT
< ETag: "85-65afe18dfa2c"
< Accept-Ranges: bytes
< Content-Length: 133
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: OJE DOMINION MAYOWA</p><p>Reg No: FWS02511507</p></body></html>
*d*s*BE4i*PN*FQKL*
ph*vx*E*d*s*BE4i*PN*FQKL*
```

Fig 16: Confirmed the creation of the captured file

```

(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -r evidence/basic.pcapng
1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 49640 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1885922403 TSecr=0 WS=256
2 0.000191629 127.0.0.1 → 127.0.0.1 TCP 74 80 → 49640 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=2017850036 TSecr=1885922403 WS=256
3 0.000219975 127.0.0.1 → 127.0.0.1 TCP 66 49640 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1885922403 TSecr=2017850036
4 0.000457005 127.0.0.1 → 127.0.0.1 HTTP 149 GET /basic.html HTTP/1.1
5 0.001238496 127.0.0.1 → 127.0.0.1 TCP 66 80 → 49640 [ACK] Seq=1 Ack=84 Win=65536 Len=0 TSval=2017850037 TSecr=1885922403
6 0.019564630 127.0.0.1 → 127.0.0.1 HTTP 450 HTTP/1.1 200 OK (text/html)
7 0.019627885 127.0.0.1 → 127.0.0.1 TCP 66 49640 → 80 [ACK] Seq=84 Ack=385 Win=65280 Len=0 TSval=1885922422 TSecr=2017850055
8 0.019878196 127.0.0.1 → 127.0.0.1 TCP 66 49640 → 80 [FIN, ACK] Seq=84 Ack=385 Win=65536 Len=0 TSval=1885922423 TSecr=2017850055
9 0.020610210 127.0.0.1 → 127.0.0.1 TCP 66 80 → 49640 [FIN, ACK] Seq=385 Ack=85 Win=65536 Len=0 TSval=2017850056 TSecr=1885922423
10 0.021538510 127.0.0.1 → 127.0.0.1 TCP 66 49640 → 80 [ACK] Seq=85 Ack=386 Win=65536 Len=0 TSval=1885922423 TSecr=2017850056

(kali@kali)-[~/SBT-DF203-Lab1]
$

```

Fig 17: Confirmation of receiving packets

Then we create a copy of the original **basic.pcapng** to preserve the evidence while working on it

```

(kali@kali)-[~/SBT-DF203-Lab1]
$ cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab1]
$ ls -lh working/basic_working.pcapng
-rw-r--r-- 1 kali kali 1.8K Sep  8 21:48 working/basic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab1]
$

```

Fig 18: Creating a copy of the original evidence

Next, we hash it to verify the integrity of both the original evidence and the copy

```

(kali@kali)-[~/SBT-DF203-Lab1]
$ sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
6d622f6162b5b9057d929eb3b0ecdcb06ed3a85fe727cce6434bc17391f75043 evidence/basic.pcapng
6d622f6162b5b9057d929eb3b0ecdcb06ed3a85fe727cce6434bc17391f75043 working/basic_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab1]
$

```

Fig 19: Checking the hash values of both the original and the copy

Part C - Analyze the TCP Three-Way Handshake

We run the tshark to be able to get the three handshake by analyzing the TCP for SYN, SYN-ACK and ACK.

```

(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 66 tcp.flags.syn==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
1 2026-09-08T21:48:23.986673290+0100 127.0.0.1 49640 127.0.0.1 80 0x0002 0 0
2 2026-09-08T21:48:23.986864919+0100 127.0.0.1 80 127.0.0.1 49640 0x0012 0 1

(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 66 tcp.flags.syn==0 66 tcp.flags.ack==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | head -n 1 | tee -a reports/handshake.tsv
3 2026-09-08T21:48:23.986893265+0100 127.0.0.1 49640 127.0.0.1 80 0x0010 1 1

(kali@kali)-[~/SBT-DF203-Lab1]
$

```

Fig 20: Checking the hash values of both the original and the copy

Packet	Flags	Client Seq	Server Seq/ACK	Timestamp	Interpretation
1	SYN	Seq = 0	Ack = 0	21:48:23.986673290+0100	Client requests a TCP session.
2	SYN, ACK	Seq = 0	Ack = 1	21:48:23.986864919+0100	Server acknowledges and provides its initial sequence number.
3	ACK	Seq = 1	Ack = 1	21:48:23.986893265+0100	Client acknowledges; the connection becomes established.

Part D - Examine the HTTP Request and Response

We then examine the HTTP Request and Response of the packets sent on the port

```

(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host -e http.user_agent \
  | tee reports/http_requests.tsv
4      2026-09-08T21:48:23.987130295+0100      127.0.0.1      49640      127.0.0.1      80      GET      /basic.html      127.0.0.1      curl/8.21.0
(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e http.response.code -e http.response.phrase \
  -e http.server -e http.content_type -e http.content_length \
  | tee reports/http_responses.tsv
6      2026-09-08T21:48:24.006237920+0100      200      OK      Apache/2.4.68 (Debian)      text/html      133
(kali@kali)-[~/SBT-DF203-Lab1]
$

```

Fig 21: Examining the HTTP Request and Response

GET method = /basic.html

URI Host = 127.0.0.1

User-Agent = curl/8.21.0

Status Code = 200 / OK,

Server = Apache/2.4.68 (Debian),

Content - Type = text/html

```
(kali㉿kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0' -T fields \
  -e frame.number -e tcp.seq -e tcp.ack -e tcp.len -e tcp.flags \
  | tee reports/seq_ack_progression.txt
1      0      0      0      0x0002
2      0      1      0      0x0012
3      1      1      0      0x0010
4      1      1      83      0x0018
5      1      84      0      0x0010
6      1      84      384      0x0018
7      84      385      0      0x0010
8      84      385      0      0x0011
9      385      85      0      0x0011
10     85      386      0      0x0010

(kali㉿kali)-[~/SBT-DF203-Lab1]
$
```

Fig 22: Checking the Progression

Looking at the seq/ack numbers alongside the packet lengths, I could actually see how TCP keeps track of data as it moves between the client and server.

For the first three packets (the handshake: SYN, SYN-ACK, ACK), the length is 0 because no actual data is being sent yet, just the initial setup of the connection.

Then in packet 4, the client sends the HTTP GET request, which is 83 bytes long. Right after, in packet 5, I noticed the server's ack number was 84, which makes sense once I realized it's just the client's starting sequence number (1) plus the 83 bytes it just received. So the ack number is basically the server saying "I got everything up to byte 84."

The same pattern happens in the other direction: in packet 6, the server sends back its response (the HTML page), which is 384 bytes. In packet 7, the client's ack becomes 385, again matching the server's starting sequence (1) plus the 384 bytes sent.

What this told me is that ack equals previous seq plus previous length, every single time. That's how TCP confirms nothing was lost or corrupted along the way; every byte sent gets accounted for in the next acknowledgement.

The last few packets (8 to 10) are the connection closing down with FIN/ACK, and even those follow the same plus 1 pattern since a FIN counts as one byte for sequencing purposes.

Overall, this was a good way for me to actually see something I'd only read about before: how TCP's reliability really works at the byte level, not just in theory.

Part E - Inspect Encapsulation and Connection Closure

```
(kali@kali)-[~/SBT-DF203-Lab1]
$ 
(kali@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
8      2026-09-08T21:48:24.006551486+0100      127.0.0.1      49640      127.0.0.1      80      0x0011      84      385
9      2026-09-08T21:48:24.007283500+0100      127.0.0.1      80      127.0.0.1      49640      0x0011      385      85
(kali@kali)-[~/SBT-DF203-Lab1]
$
```

Fig 23: Shows the ending and closing connection of the FIN/ACK Packets

```
kali@kali: ~/SBT-DF203-Lab1
$ tshark -r working/basic_working.pcapng -Y 'frame.number==4' | tee reports/frame4_expanded.txt
Frame 4: Packet, 149 bytes on wire (1192 bits), 149 bytes captured (1192 bits) on interface lo, id 0
  Section number: 1
    Interface id: 0 (lo)
      Interface name: lo
      Interface description: Loopback
      Encapsulation type: Ethernet (1)
      Arrival Time: Sep  8, 2026 21:48:23.987130295 WAT
      UTC Arrival Time: Sep  8, 2026 20:48:23.987130295 UTC
      Epoch Arrival Time: 1788900503.987130295
      [Time shift for this packet: 0.000000000 seconds]
      [Time delta from previous captured frame: 237.030 microseconds]
      [Time since reference or first frame: 457.005 microseconds]
      Frame Number: 4
      Frame Length: 149 bytes (1192 bits)
      Capture Length: 149 bytes (1192 bits)
      [Frame is marked: False]
      [Frame is ignored: False]
      [Protocol: in Frame: ethertype:ip:tcp:http] | tshark -r working/basic_working.pcapng
      Character encoding: ASCII (0)
      Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
        Destination: 00:00:00:00:00:00 (00:00:00:00:00:00)
          .... 0. .... = LG bit: Globally unique address (factory default)
          .... 0. .... = IG bit: Individual address (unicast)
        Source: 00:00:00:00:00:00 (00:00:00:00:00:00)
          .... 0. .... = LG bit: Globally unique address (factory default)
          .... 0. .... = IG bit: Individual address (unicast)
        Type: IPv4 (0x0800)
        [Stream index: 0]
      Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
        0100 .... = Version: 4
        .... 0101 = Header Length: 20 bytes (5)
        Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... 00.. = Explicit Congestion Notification: Not ECN-Capable Transport (0)
      Total Length: 135
      Identification: 0x69a5 (27045)
      010. .... = Flags: 0x2, Don't Fragment
      0 .... = Reserved bit: Not set
      .1.. .... = Don't fragment: Set
```

Fig 24: Listing out the Frame length, IP total Length, TCP header and TCP Payload

Que: Explain the relationship among frame length, IP total length, TCP header length and TCP payload length.

Each layer adds its own header on top of the payload from the layer above it. The Frame length includes everything (Ethernet + IP + TCP + HTTP data). If we remove the Ethernet header and you would get the IP total length. If we remove the IP header you get the TCP segment. If we remove the TCP header, we are left with just the actual HTTP data being carried. Every layer "wraps" the one above it.

Que: Locate FIN/ACK packets or a reset that closes the session. Explain the observed termination sequence.

Packet 8 carries flags 0x0011 (FIN, ACK), which is the server telling the client it has finished sending data and is ready to close its side of the connection, while also acknowledging the data it had already received.

Packet 9 also shows flags 0x0011 (FIN, ACK), this time from the client, acknowledging the server's FIN and sending its own FIN to close its side of the connection too.

Packet 10 shows flags 0x0010 (ACK only), which is the final acknowledgement confirming both sides agreed the connection is fully closed.

No RST packets were observed anywhere in the capture, which means the connection ended properly rather than being forcibly terminated, a strong indicator that the HTTP transaction completed successfully with no errors or unexpected drops.

Que:

Loopback (lo) traffic never touches an actual network interface card or physical medium, so there is no need for Ethernet framing or MAC addressing at all. Data goes directly through the kernel's network stack from one local socket to another, skipping the data-link layer entirely. That's why Wireshark shows something like 00:00:00_00:00:00 or no Ethernet header at all for loopback captures. To capture genuine Ethernet-layer info (real source/destination MAC addresses), you need traffic to actually cross a real or virtual NIC, which is why the lab asks you to repeat the capture between two separate VMs on a host-only network.

Required Forensic Findings

Finding	Value/Observation
Capture start and end time	Started 2026-09-08 21:48:23.986673290 (packet 1, SYN); It ended shortly after with the FIN/ACK teardown (packets 8–10), all within the same second, the total capture duration was under 1 millisecond
Client IP and port	127.0.0.1, port 49640
Server IP and port	127.0.0.1, port 80
Client initial sequence number	0 (relative sequence number as shown by tshark)
Server initial sequence number	0 (relative sequence number)
HTTP request timestamp	2026-09-08 21:48:23.987130295

Finding	Value/Observation
Requested URI	/basic.html
HTTP response code	200 OK
Content type and length	text/html, 133 bytes
Connection termination method	good termination via FIN/ACK exchange (packets 8–9 showed flags 0x0011, confirming FIN, ACK), not an abrupt RST
MAC-address observation	Source and destination MAC addresses both showed as 00:00:00_00:00:00 — expected on loopback, since traffic never leaves the kernel to touch a real network interface, so no genuine Ethernet framing/MAC addressing occurs

This lab gave me hands-on insight into how HTTP traffic is structured, transmitted, and closed at the packet level, and how each protocol layer builds on the one before it. More importantly, it introduced me to real forensic practice, capturing evidence carefully, verifying its integrity through hashing, and documenting findings in a professional way. These skills are foundational to a good digital forensics trainee or expert.